DEBUGGING · DOMAIN 9 - DEBUGGING AND DEPLOYING

Diagnosing failures

Section 9. Five places to look, and knowing which one before you start.

SECTION 9 of 10 WEIGHT 10% TOPIC Debugging

admin@sthithapragna.com

Each source answers a different question. **Choosing the wrong one wastes an afternoon before you learn anything.**

Five sources, five questions

SOURCE	WHAT IT ANSWERS
Spark UI	Task level. Skew, spill, slow stages, the physical plan.
Cluster logs	Driver level. Out of memory, init script failures, dependency errors.
Cluster event log	Lifecycle. Did it start, resize, terminate, or stop responding?
System tables	Historical and cross-workspace. Run history, cluster events, audit.
Query profile	SQL warehouse. Operator timing, pruning, join strategy.
Pipeline event log	Dataset level. Flow progress, expectations, data quality.

ON THE EXAM

Learn this table as a routing decision. When something fails, the first question is which row you are in.

Where your own code's output lands

- **Print and log statements from notebooks, jobs and libraries go to the Spark driver logs.**
- So your stack traces, your Python exceptions and your own logging are all here.
- Available from the **Driver logs tab** on the compute details page. Click a file name to download it.
- By default, **only users with CAN MANAGE can view them**, which is module 24's secret redaction warning.

> ON THE EXAM
>
> If your code printed it, it is in the driver logs. That single rule resolves most 'where do I look' questions about application errors.

Where the executors' problems land

- Worker-side errors do **not** appear in the driver logs. Task failures and executor out-of-memory errors are here.
- View them through the **Spark UI tab**, or in the delivered log location if you configured one.
- **Log delivery generates a log4j file only in the driver folder, not in executor folders.** That is by design.
- Executor folders carry standard output and standard error instead.

> ON THE EXAM
>
> The missing log4j file in executor folders is not a bug. Knowing that saves you investigating a delivery problem you do not have.

What happened to the machine

- The compute event log records **lifecycle events**: creation, termination, resizing, configuration edits.
- Click a row and then the JSON tab for the full detail of an event.
- **Events are stored for 60 days**, comparable to other retention windows on the platform.
- This is where a **driver not responding** event appears, often with a message about garbage collection.

> ON THE EXAM
>
> If the question is 'did it even start', or 'why did it resize', this is the source. Nothing else records those.

The silent failure

- An init script that fails can leave a cluster that **appears to start** while every job on it fails immediately.
- Init script logs are a **separate category**, delivered to their own folder when log delivery is configured.
- Library installation and environment setup failures land here, not in the driver logs.
- Without log delivery configured, diagnosing an init script failure is considerably harder.

> ON THE EXAM
>
> A cluster that starts and then fails everything is the signature. Look at the init script logs before reading a single stack trace.

The Spark UI dies with the cluster

- **The Spark UI is unavailable once a cluster is terminated.**
- Job compute typically terminates immediately after the run, which is exactly when you want to investigate.
- So **configure log delivery** to a storage location, and the logs survive the cluster.
- Delivered structure: a folder per cluster ID, containing driver, executor, event log and init script folders.

> ON THE EXAM
>
> This is the argument for configuring log delivery in advance. After the failure, the evidence has already gone.

The compute metrics UI

- Available for **classic all-purpose and jobs compute**, on the **Metrics tab** of the compute page.
- **Metrics are collected every minute**, and historical ranges are selectable with a date picker.
- Useful for memory pressure over time, which a single stack trace cannot show you.
- This replaced Ganglia, which is not available from DBR 13 onward. Older material still references it.

> ON THE EXAM
>
> An error tells you the moment something broke. Metrics tell you whether it had been degrading for an hour beforehand.

The four documented causes

- The **driver instance type is not optimal** for the load being placed on it.
- **Memory-intensive operations executed on the driver** rather than distributed.
- **Many notebooks or jobs running in parallel** on the same cluster, all sharing one driver.
- **Heavy Python logic** such as loops, UDFs and non-Spark operations, **which runs exclusively on the driver**.

> ON THE EXAM
>
> Three of the four are about work landing on the driver that should have been distributed. That is the pattern to recognise.

What driver memory pressure looks like

- The driver **crashes and restarts**, or becomes unresponsive through frequent full garbage collection.
- A **driver not responding** event appears, often saying the driver is up but unresponsive, likely due to GC.
- A notebook reports that the Spark driver stopped unexpectedly and will be reattached.
- The classic code cause is **collecting a large result to the driver** rather than keeping it distributed.

> ON THE EXAM
>
> Notice that unresponsive and crashed are the same underlying problem. Garbage collection thrashing is memory pressure that has not finished failing yet.

Out of memory and spill are not the same

- **Spill** means data did not fit in memory and was written to disk. The job continues, more slowly.
- **Out of memory** means it could not continue at all. The task fails and may take the executor with it.
- Spill on **most** tasks is memory pressure. Spill on a **few** is skew, from module 18.
- Both are worker-side, so both are in the **executor logs** rather than the driver logs.

> ON THE EXAM
>
> Spill is a performance symptom that a query profile and the Spark UI diagnose. Out of memory is a failure, and it is in the logs.

When the evidence is already gone

- System tables persist and are queryable, where the Spark UI is ephemeral and per run.
- Job run history, cluster events and audit records support **trend analysis** across the whole estate.
- Remember the retention differences from module 17. Node utilisation keeps only 90 days.
- And the cluster event log keeps **60 days**, which is shorter than most system tables.

> ON THE EXAM
>
> For 'has this been getting worse for months', system tables are the only source. Nothing else keeps that long a memory.

An order to work in

- **Did it start?** Cluster event log, and init script logs if it started and then failed everything.
- **Did the code fail?** Driver logs for the exception and the stack trace.
- **Did a task fail?** Executor logs, then the Spark UI for the pattern across tasks.
- **Was it just slow?** Query profile or Spark UI, and metrics for what was happening over time.

> ON THE EXAM
>
> Four questions in order. Most wasted debugging time is spent answering the third question with the tools for the first.

What to carry into the exam

- **Your code's output is in the driver logs. Worker errors are in the executor logs.** That split is examinable directly.
- **The Spark UI dies with the cluster.** Configure log delivery in advance.
- An init script failure looks like a cluster that starts and then fails everything.
- Driver OOM is usually work that should have been distributed. Watch for parallel notebooks sharing one driver.
- Event log keeps **60 days**. Metrics are collected **every minute**. System tables are for trends.

> ON THE EXAM
>
> When a question names a symptom, name the source before choosing an answer. Most distractors are the right technique applied to the wrong layer.

> A nightly job on job compute failed at 3am. At 9am an engineer opens the run and finds the Spark UI is unavailable. No log delivery is configured on the cluster.
>
> They need to know which stage failed and why.

What is the situation?

A The Spark UI is unavailable because the cluster terminated. Without log delivery, that detail is gone; configure it before the next run.

B The Spark UI takes several hours to become available after a run; check again later.

C The Spark UI requires CAN MANAGE, which the engineer lacks.

D Restart the terminated cluster, which restores its Spark UI with the previous run's data.

◆ PAUSE AND DECIDE

What is the situation?

A The Spark UI is unavailable because the cluster terminated. Without log delivery, that detail is gone; configure it before the next run.

The UI does not survive termination, and job compute terminates immediately after the run.

B The Spark UI takes several hours to become available after a run; check again later.

There is no such delay. It is gone with the cluster.

C The Spark UI requires CAN MANAGE, which the engineer lacks.

That gate applies to driver logs. This is an availability problem, not permissions.

D Restart the terminated cluster, which restores its Spark UI with the previous run's data.

Restarting re-creates the compute with the same ID. It does not restore the previous run's UI.

ON THE EXAM

An unsatisfying answer, and the honest one. The remedy is preventive, which is exactly why log delivery is worth configuring before you need it.

> A cluster starts successfully and reports healthy. Every job submitted to it fails within seconds with import errors for a library the team installed via an init script.
>
> The driver logs show only the import failures.

Where should they look next?

A The executor logs, since library installation happens on the workers.

B The init script logs, since a failed init script leaves a cluster that starts but cannot run anything.

C The compute metrics UI, to check whether memory prevented installation.

D The query profile, to see which operator is failing.

◆ PAUSE AND DECIDE

Where should they look next?

A The executor logs, since library installation happens on the workers.

Would show the same downstream symptom, not the cause.

B The init script logs, since a failed init script leaves a cluster that starts but cannot run anything.

The signature failure mode. Init script output is a separate log category with its own folder.

C The compute metrics UI, to check whether memory prevented installation.

Metrics show resource usage over time, not setup failures.

D The query profile, to see which operator is failing.

Wrong layer entirely. Nothing has reached a query.

ON THE EXAM

The import error is the symptom, and the driver logs faithfully report it. The cause is one layer earlier, at cluster startup.

> A shared all-purpose cluster supports eight analysts. Through the morning it becomes unresponsive, and the event log shows a driver not responding event mentioning garbage collection. Individual notebooks report the driver stopped unexpectedly.
>
> No single notebook is processing an unusually large dataset.

What is the most likely cause?

A. Data skew in one analyst's query is overloading a single executor.

B. Many notebooks running in parallel on the same cluster share one driver, which is a documented cause of driver memory pressure.

C. Shuffle spill has filled the driver's local disk.

D. The cluster needs more worker nodes.

◆ PAUSE AND DECIDE

What is the most likely cause?

A Data skew in one analyst's query is overloading a single executor.

Skew is a worker-side problem. The symptom here is on the driver.

B Many notebooks running in parallel on the same cluster share one driver, which is a documented cause of driver memory pressure.

One of the four documented causes, and it explains why no single notebook looks unusual.

C Shuffle spill has filled the driver's local disk.

Shuffle spill occurs on executors, and it is a disk and performance issue rather than driver GC.

D The cluster needs more worker nodes.

Adding workers does not relieve a driver that is out of memory.

ON THE EXAM

The clue is that no single notebook is unusual. Each analyst's work is reasonable, and the driver is carrying all eight at once.

> A job fails with an out-of-memory error. An engineer opens the driver logs, finds nothing but ordinary progress messages, and concludes the log delivery is broken.
>
> The error message referenced a task failure during a large shuffle.

What should they be told?

A. Log delivery is broken and needs reconfiguring.

B. Worker-side out-of-memory errors appear in the executor logs, not the driver logs.

C. Out-of-memory errors are only visible in the compute metrics UI.

D. The driver logs are truncated after a fixed size and the error was dropped.

◆ PAUSE AND DECIDE

What should they be told?

A Log delivery is broken and needs reconfiguring.
Nothing is broken. They are reading the wrong log.

B Worker-side out-of-memory errors appear in the executor logs, not the driver logs.
The driver and executor log split, and a task failure during a shuffle is unambiguously worker-side.

C Out-of-memory errors are only visible in the compute metrics UI.
Metrics show memory over time. The error itself is in the logs.

D The driver logs are truncated after a fixed size and the error was dropped.
Invented, and it would not explain a worker-side error being absent from a driver log.

ON THE EXAM

The phrase task failure is the giveaway. Tasks run on executors, so their failures are recorded there.

> A team is asked whether a nightly job's runtime has been degrading over the past six months, and whether cluster memory pressure has been increasing alongside it.
>
> The clusters are job compute and terminate after each run.

Which sources can answer each half?

A The Spark UI for both, comparing runs across the period.

B System tables for the run history over six months; node utilisation retains only 90 days, so the memory half is partly unanswerable.

C The cluster event log for both, since it retains a full year.

D The compute metrics UI for both, using the date picker across six months.

◆ PAUSE AND DECIDE

Which sources can answer each half?

A The Spark UI for both, comparing runs across the period.

Ephemeral and per run, and these clusters terminate.

B System tables for the run history over six months; node utilisation retains only 90 days, so the memory half is partly unanswerable.

Run history is retained a year; node utilisation is one of the 90-day exceptions from module 17.

C The cluster event log for both, since it retains a full year.

The event log retains 60 days, and it records lifecycle events rather than memory.

D The compute metrics UI for both, using the date picker across six months.

Metrics are per compute, and these clusters no longer exist.

ON THE EXAM

Two halves, two answers, and one of them is partly no. Retention decides which questions are answerable at all.

> A pipeline's silver streaming table stopped updating overnight. The pipeline itself reports a failure, and an engineer begins by opening the Spark UI to look for a slow stage.
>
> The failure message mentions an expectation.

What should they open instead?

A. The pipeline event log, which records flow progress and data quality at dataset level.

B. The driver logs, since pipeline failures always surface as Python exceptions.

C. The cluster event log, since the pipeline's compute must have failed.

D. The query profile, since the pipeline runs SQL.

◆ PAUSE AND DECIDE

What should they open instead?

A The pipeline event log, which records flow progress and data quality at dataset level.

An expectation is a pipeline-level construct, and the event log is where its results and failures are recorded.

B The driver logs, since pipeline failures always surface as Python exceptions.

Some do. An expectation violation is reported by the pipeline, at dataset level.

C The cluster event log, since the pipeline's compute must have failed.

Nothing suggests the compute failed. The pipeline reported the failure itself.

D The query profile, since the pipeline runs SQL.

Reachable from the pipelines UI, and it answers performance rather than an expectation failure.

ON THE EXAM

The word expectation places the failure at dataset level. Opening the Spark UI answers a performance question nobody asked.